

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

Data Interpretation

Code of Conduct:

I Cheredla Jasvina/192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	

Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	
---------	-----	--	-----------------------------------	-----------------------------------	---	--

Annexure A

Data Interpretation

1: Hough Transform & Shape Detection

Scenario: Detect linear features such as roads, rivers, and canals in high-resolution satellite images.

Answer: The Hough Transform is a feature-detection technique used to identify geometric shapes, especially straight lines, from an image. It is suitable for satellite images because roads, rivers, and canals often appear as long linear structures. The method can detect lines even when they are partially broken because of noise or occlusion.

1. Image Preprocessing: First, read the grayscale satellite image. Since the image may contain noise caused by clouds, shadows, sensor errors, or other objects, apply Gaussian or median filtering. Histogram equalization can be used to improve contrast between linear features and their surroundings.

2. Edge Detection: Apply the Canny edge detector to the preprocessed image. Canny detects strong intensity changes and produces an edge image containing possible boundaries of roads, rivers, and canals.

3. Region Selection: If necessary, restrict processing to relevant regions of the satellite image. Morphological operations can also be used to remove very small isolated noise regions.

4. Hough Transform: Apply the Hough Transform to the edge image. Each edge point votes for possible lines in parameter space. A straight line can be

represented by:

Here, ρ is the perpendicular distance from the origin to the line and θ is the angle of the perpendicular.

5. Finding Lines: The Hough accumulator stores the number of votes for different combinations. Peaks in the accumulator indicate strong linear structures.

6. Handling Multiple Features: Since the image can contain several roads, rivers, and canals, multiple peaks can be selected from the accumulator. Each peak represents a possible linear feature.

7. Handling Intersections: At road intersections, several lines can cross at the same location. The Hough Transform can detect each line independently because they have different values of ρ and θ .

8. Handling Partial Occlusion: A complete line is not required. Even if part of a road or river is hidden by clouds, buildings, trees, or shadows, the visible edge points can still generate enough votes to identify the line.

9. Removing False Positives: Short lines and weak detections should be removed using thresholds based on line length, number of votes, angle, and position.

10. Curved Features: Standard Hough Transform works best for straight lines. For curved roads or rivers, probabilistic Hough methods, generalized Hough Transform,

polynomial curve fitting, or contour-based methods can be used.

Pseudocode:

Algorithm Hough_Linear_Feature_Detection

Input: Grayscale satellite image I

Output: Detected roads, rivers, and canals

- Read satellite image I
- Convert I to grayscale if required
- Normalize image intensity
- Apply Gaussian/median filtering
- Apply histogram equalization
- Apply Canny edge detection
- Remove small noisy edge regions
- Define relevant region of interest
- Apply Hough Transform
- Build Hough accumulator
- Find strong peaks in accumulator
- Convert peaks into image-space lines
- Remove short and weak line detections
- Merge nearby line segments
- Draw detected lines on original image
- Display final result
- End

Expected Output: The output is the original satellite image with detected roads, rivers, and canals highlighted using colored lines.

Advantages: It can detect multiple lines, tolerate partial occlusion, and work even when the line is broken.

Limitations: It may produce false detections in images with many edges and is less suitable for highly curved features.

Question 2: PCA & Shape Priors for Object Recognition

Scenario: Segment organs such as the liver or heart from MRI scans using shape priors.

Answer: PCA-based shape modeling learns the common shape and variation of an organ from annotated training masks. The learned shape model is then used as a prior to guide segmentation of new MRI images, particularly when boundaries are noisy or partially missing.

1. Training Mask Preparation: Read the 50–100 annotated organ masks. Each mask contains the correct boundary of the liver, heart, or other organ.

2. Shape Alignment: Align the organ shapes to reduce differences caused by translation, rotation, and scaling. Alignment allows PCA to focus on actual anatomical shape variation.

3. Shape Representation: Convert each organ mask into a numerical shape vector. Boundary coordinates or landmark points can be used for this representation.

4. Mean Shape: Calculate the average of all aligned training shapes. This produces the mean organ shape.

5. PCA: Apply Principal Component Analysis to the training shape vectors. PCA identifies the most important directions in which the organ shape varies.

6. Shape Model: The shape can be represented approximately as:

where μ is a possible organ shape, Σ is the mean shape, contains the principal components, and contains the shape parameters.

7. Test Image Preprocessing: Read a new MRI image and apply noise reduction, intensity normalization, and contrast enhancement.

8. Initial Segmentation: Obtain an initial estimate of the organ boundary using thresholding, edge detection, region growing, active contours, or another segmentation technique.

9. Shape Prior Fitting: Compare the initial segmentation with the PCA shape model. Adjust the boundary so that it follows both image information and the learned anatomical shape.

10. Partial Boundary Handling: If a portion of the organ boundary is missing or unclear because of noise, the PCA model can estimate the missing portion based on the shapes learned from the training set.

11. Final Segmentation: Combine image information with the shape prior and generate the final binary organ mask.

Pseudocode:

Algorithm PCA_Organ_Shape_Segmentation

Training:

1. Read annotated organ masks
2. Extract organ boundaries
3. Align all organ shapes
4. Convert shapes into vectors
5. Calculate mean shape
6. Calculate covariance matrix
7. Apply PCA
8. Select important principal components
9. Construct PCA shape model

Testing:

10. Read test MRI image
11. Apply noise reduction
12. Normalize intensity
13. Enhance image contrast
14. Locate approximate organ region
15. Generate initial segmentation
16. Fit initial boundary to PCA shape model
17. Apply shape constraints
18. Correct missing or noisy boundaries
19. Generate final organ mask
20. Display segmented organ

End

Advantages: PCA reduces dimensionality, provides a compact shape representation, and prevents segmentation from taking unrealistic shapes.

Limitations: PCA assumes mainly linear shape variation and may not represent very

unusual anatomical structures. It also depends on the quality and diversity of the training masks.

Expected Output: A binary or colored segmentation mask showing the detected organ region on the MRI image.

Question 3: Machine Learning Methods (SVM, HMM, GMM, EM)

Scenario: Recognize human activities such as walking, running, and jumping from video feature trajectories.

Answer: Human activity recognition can use joint trajectories or optical-flow features extracted from video frames. SVM can classify activities, HMM can model their temporal structure, GMM can represent feature distributions, and EM can estimate the parameters of the GMM.

1. Video Acquisition: Read the training videos at 30–60 frames per second.
2. Human Feature Extraction: Detect the human body and extract joint coordinates such as the head, shoulders, elbows, hands, hips, knees, and feet.
3. Optical Flow: Alternatively, optical flow can be calculated to represent the direction and magnitude of motion between consecutive frames.
4. Feature Construction: Calculate motion features such as joint displacement, velocity, acceleration, joint angles, distances between joints, and movement direction.
5. Normalization: Normalize coordinates with respect to body size and position. This makes the system less sensitive to differences in person size or camera distance.
6. GMM: A Gaussian Mixture Model represents the probability distribution of motion features using several Gaussian components. Different components can represent different motion patterns.
7. EM Algorithm: The Expectation-Maximization algorithm estimates the parameters of the GMM. During the E-step, probabilities of observations belonging to Gaussian components are calculated. During the M-step, the means, variances, and mixture weights are updated.
8. HMM: A Hidden Markov Model represents an activity as a sequence of hidden states. For example, walking contains a sequence of repeated leg and arm movement states.
9. SVM: An SVM can classify extracted feature vectors into categories such as walking, running, or jumping. It learns a decision boundary between activity classes.
10. Testing: Extract features from an unknown video and provide them to the trained models.
11. Final Decision: Combine the classification or probability results and select the activity with the highest confidence.

Role of Methods:

SVM: Supervised classifier for distinguishing activity classes.

HMM: Models the temporal sequence of movements.

GMM: Represents the probability distribution of feature vectors.

EM: Estimates the parameters of the GMM.

Pseudocode:

Algorithm Human_Activity_Recognition

Training:

1. Read training videos
2. Extract human joint positions

3. Calculate optical-flow features if required
4. Calculate motion features
5. Normalize feature vectors
6. Initialize GMM parameters
7. Perform EM algorithm
8. Train GMM
9. Train HMM using temporal activity sequences
10. Train SVM using activity labels

Testing:

11. Read test video
12. Extract joint or optical-flow features
13. Calculate motion features
14. Normalize features
15. Calculate GMM likelihood
16. Evaluate HMM sequence probability
17. Apply trained SVM
18. Compare activity scores
19. Select highest-confidence activity
20. Output activity label

End

Expected Output: The system may output Walking, Running, Jumping, or another trained activity class.

Advantages: HMM handles temporal information effectively, SVM provides strong classification, and GMM/EM can model complex feature distributions.

Limitation: Training can become computationally expensive when there are many activities and long video sequences.

Question 4: Deep Learning (CNNs, Transformers, GANs)

Scenario: Detect rare obstacles such as traffic cones and animals in urban self-driving car images.

Answer: Since only 500–1000 real labeled images are available, transfer learning, data augmentation, and possibly GAN-generated images can be used to improve the object detector.

1. Dataset Preparation: Collect RGB images and bounding-box annotations for all known obstacle classes.
2. Image Preprocessing: Resize the images, normalize pixel values, and ensure that bounding boxes are adjusted correctly after resizing.
3. Data Augmentation: Apply horizontal flipping, scaling, cropping, rotation, brightness changes, contrast changes, and simulated weather conditions. This creates more variation from the limited dataset.
4. CNN-Based Detection: Use a pretrained CNN-based object detector. The pretrained network already contains useful visual features learned from large datasets.
5. Transfer Learning: Replace or adapt the final detection layers and fine-tune the network using the obstacle dataset.

6. Transformer-Based Detection: A Vision Transformer or transformer-based object detector can also be used. It uses attention mechanisms to learn relationships between different image regions.
7. GAN-Based Augmentation: A GAN can generate synthetic examples of rare obstacles. These examples can be added to the training dataset after checking their quality.
8. Training: Train the detector using real and carefully selected synthetic images. Use validation data to monitor performance and prevent overfitting.
9. Object Detection: For a new image, the trained detector predicts the obstacle class, bounding-box coordinates, and confidence score.
10. Non-Maximum Suppression: Multiple overlapping boxes for the same obstacle are reduced to one final detection.
11. Real-Time Processing: For autonomous driving, a lightweight detector and optimized inference should be used to satisfy real-time requirements.

Pseudocode:

Algorithm Rare_Obstacle_Detection

1. Load labeled RGB images
2. Load bounding-box annotations
3. Resize images
4. Normalize pixel values
5. Apply data augmentation
6. Load pretrained CNN detector
7. Apply transfer learning
8. Fine-tune detector on obstacle dataset
9. If required, generate synthetic images using GAN
10. Validate synthetic images
11. Add useful synthetic images to training data
12. Train final detector
13. Read test image
14. Extract image features
15. Detect obstacle objects
16. Remove low-confidence detections
17. Apply Non-Maximum Suppression
18. Draw bounding boxes
19. Display class labels and confidence scores

End

CNN: Good for efficient image feature extraction and real-time object detection.

Vision Transformer: Uses self-attention and can capture long-range relationships between image regions, but may require greater computational resources.

GAN: Generates additional synthetic training examples and is especially useful for rare obstacle classes.

Expected Output: The system displays bounding boxes around objects such as traffic cones or animals along with their class labels and confidence scores.

Main Challenge: Rare obstacles have very few examples, so the system must avoid overfitting and should be trained with diverse real and synthetic examples.

Question 5: Graph-Based Learning for Shape Correspondence

Scenario: Find correspondences between 3D human face meshes with different numbers of vertices.

Answer: Graph-based learning represents each 3D face as a graph and learns relationships between vertices. This allows corresponding facial points to be identified even when different meshes have different vertex counts or structures.

1. Load 3D Meshes: Read the OBJ or PLY face meshes. Each mesh contains vertices and edges/faces describing the 3D facial surface.
2. Graph Construction: Convert each face mesh into a graph. Each vertex becomes a graph node, and connections between neighboring vertices become graph edges.
3. Vertex Features: Extract features such as 3D coordinates, surface normals, curvature, local neighborhood information, and geometric descriptors.
4. Edge Features: Edge length, relative orientation, and neighborhood relationships can also be stored.
5. Training Data: Use the 20–50 training face meshes with annotated landmark correspondences. Examples include the nose tip, eye corners, mouth corners, and chin.
6. Graph Neural Network: A graph neural network can learn useful representations from the node and edge relationships.
7. Learning Correspondences: During training, the network learns that similar facial structures should have similar representations even if their vertex coordinates or mesh structures differ.
8. Testing: For two unseen face meshes, generate graph representations and calculate feature embeddings for their vertices.
9. Similarity Calculation: Compare vertex embeddings between the two faces. Vertices with similar geometric representations receive high correspondence scores.
10. Geometric Constraints: Apply constraints based on distances, local neighborhoods, curvature, and facial structure to remove incorrect matches.
11. Different Vertex Counts: Because the system learns relationships and geometric features instead of relying only on vertex indices, it can match faces with different numbers of vertices.
12. Output: Display corresponding landmarks or regions using the same color or connect matching points between the two meshes.

Pseudocode:

Algorithm 3D_Face_Graph_Correspondence

Training:

1. Load training 3D face meshes
2. Read annotated landmark correspondences
3. Convert each mesh into a graph
4. Extract vertex coordinates
5. Extract surface normals
6. Calculate curvature features
7. Extract edge and neighborhood features
8. Train graph-based learning model

9. Learn correspondence relationships

Testing:

10. Load two unseen face meshes
11. Convert both meshes into graphs
12. Extract vertex and edge features
13. Generate graph embeddings
14. Calculate similarity between vertices
15. Find candidate correspondences
16. Apply geometric consistency constraints
17. Remove incorrect matches
18. Select final corresponding points
19. Display corresponding landmarks

End

Advantages: Graph-based learning can handle irregular mesh structures, different vertex counts, and complex 3D geometry. It is useful for face registration, 3D morphing, shape matching, and facial analysis.

Limitations: Training requires sufficient annotated correspondence data, and processing meshes containing 50,000 vertices can be computationally expensive. Sampling or hierarchical graph processing can be used to reduce the computational cost.